MECHATRONICS SYSTEM INTEGRATION

MCTA 3203

LAB 09:

BLUETOOTH DATA INTERFACING

SECTION 1

SEMESTER 2, SESSION 2025/2026

INSTRUCTOR:

ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR. ZULKIFLI BIN ZAINAL ABIDIN

DR. -ING. WAHJU SEDIONO

DATE OF EXPERIMENT: 6 MAY 2026

DATE OF SUBMISSION: 13 MAY 2026

GROUP 3

NAME	MATRIC NO
AMIR AIMAN BIN ARIFF AFFENDI	2315925
ANIS AQILAH BINTI ABDULLAH	2329678
AZRIL ULWAN BIN MOHAMMAD IMRI	2314617

ABSTRACT

This experiment presents a wireless temperature monitoring and control system developed using an ESP32 microcontroller with built-in Bluetooth functionality. A DHT11 sensor was utilized to detect the surrounding temperature, and the measured data was transmitted to a smartphone through a Bluetooth serial terminal application. Besides monitoring temperature readings, the application also allowed users to send control commands such as “FAN ON” and “FAN OFF” to operate two LEDs that represented the fan status, where the green LED indicated ON and the red LED indicated OFF. The experiment involved integrating hardware components, establishing Bluetooth serial communication, and programming the microcontroller using the Arduino IDE. The findings demonstrated that the temperature readings were successfully transmitted in real time and that the ESP32 accurately responded to user commands for output control. Overall, the experiment proved that Bluetooth communication is an effective and low-cost solution for remote monitoring and control systems.

TABLE OF CONTENTS.....3

1.0 INTRODUCTION

The objective of this experiment is to develop a wireless monitoring and control system using Bluetooth communication between an ESP32 development board and a smartphone or computer. This setup allows the ESP32 to transmit sensor data, such as temperature or analog values, while simultaneously receiving remote control commands.

The ESP32 facilitates this communication using a 2.4 GHz radio frequency. To minimize interference, it employs Frequency-Hopping Spread Spectrum (FHSS), which involves rapid switching between 79 different channels. For data interfacing, the system utilizes the Bluetooth Classic Serial Port Profile (SPP) or the Bluetooth Low Energy (BLE) Nordic UART Service. This creates a virtual wireless bridge that functions as a standard asynchronous serial interface (UART), allowing data packets to be sent via simple serial commands and reassembled on the mobile device's terminal application.

Background Theory & Concepts:

- **Bluetooth Serial Communication**: Allows two devices to exchange data using serial-style read/write operations.
- **DHT11 Sensor**: A digital temperature and humidity sensor that communicates via a single-wire protocol.
- **Microcontroller Control Logic:** The ESP32 processes input commands and toggles digital output pins connected to LEDs.

Expected Outcome:

It was expected that:

1. The ESP32 would be able to read and transmit temperature data continuously over Bluetooth.
2. The Bluetooth terminal app would display this data in real time.
3. Commands sent from the phone would be correctly interpreted, resulting in accurate switching between ON (green LED) and OFF (red LED).

2.0 MATERIALS AND EQUIPMENTS

Here are the list of all equipments used in the experiment:

- ESP32 Development Board
- DHT11 Temperature and Humidity Sensor
- Smartphone with Bluetooth Capability
- Power Supply (USB)
- Breadboard
- Jumper Wires
- Computer with Arduino IDE installed

3.0 EXPERIMENTAL SETUP

The experimental setup was designed to establish a wireless temperature monitoring system using an ESP32, a DHT11 temperature sensor, and Bluetooth communication. The components were arranged on a breadboard and connected to allow the sensor to supply real-time temperature data to a paired external device such as a smartphone or laptop.

3.1 Hardware Configuration

1. ESP32 Microcontroller

   The microcontroller acted as the core processing unit. It supplied power to the DHT11 sensor and handled Bluetooth communication with the external device.

   - For ESP32: Built-in Bluetooth was used.

2. DH11 Temperature & Humidity Sensor

   The sensor was connected as follows:

   - VCC → 3.3V/5V pin
   - GND → Ground
   - DATA → Digital pin 4 on the microcontroller

     The sensor was placed in an open area to accurately measure the ambient temperature of the test environment.

3. Breadboard and Jumper Wires

   All components were secured on a breadboard, and jumper wires were used to establish signal, power, and ground connections.

4. Power Supply

   The ESP32 was powered using a USB cable connected to a computer, ensuring stable 5V input.

3.2 Software and Communication Setup

1. Arduino Programming

   A program was uploaded to the microcontroller to:

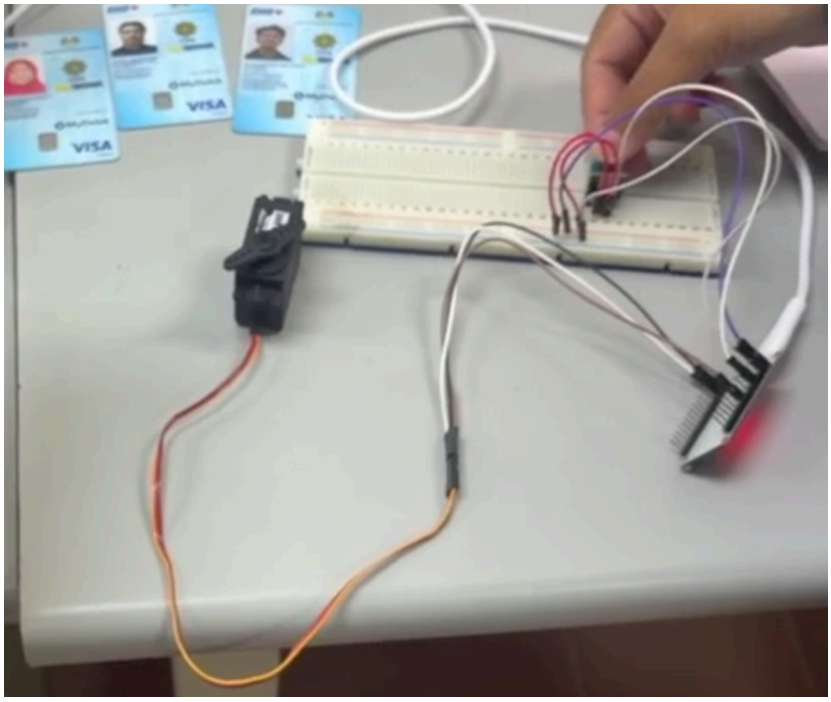
   - Read temperature data from the DHT11 sensor.
   - Send the data through the Bluetooth serial connection at 115200 baud.
   - Receive control commands (e.g., “H”, “L”) from the paired device.

2. Bluetooth Pairing

   A Bluetooth serial terminal on the smartphone was paired with the ESP32 module. The app was used to:

   - Display the real-time temperature values sent from the microcontroller.
   - Send control commands back to the system.

3.3 Setup Diagram



4.0 METHODOLOGY

The methodology outlines the procedures followed to conduct the Bluetooth-based temperature monitoring experiment. The steps include hardware preparation, software development, Bluetooth communication setup, and system testing.

4.1 Hardware Preparation

1. The DHT11 temperature sensor was placed in a test environment to measure ambient temperature.
2. All components were mounted on a breadboard, and jumper wires were used to complete power and signal pathways.
3. The microcontroller was powered via USB to ensure stable 5V supply.
4. The sensor pins were connected to the microcontroller:
    - GND to ground
    - VCC to 5V
    - DATA to digital pin 4

4.2 Software Development

Code used:

```
#include "BluetoothSerial.h"
#include <ESP32Servo.h>
#include "DHT.h"

// DHT Setup
#define DHTPIN 4     // Digital pin connected to the DHT sensor
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
```

```
BluetoothSerial SerialBT;
Servo myServo;

// Pin Definitions
const int servoPin = 18;
const int ledPin = 2;

char command;

void setup() {
Serial.begin(115200);
SerialBT.begin("ESP32_DHT_Control");

dht.begin();
 myServo.attach(servoPin);
pinMode(ledPin, OUTPUT);

 Serial.println("System Initialized. Connect your Smartphone.");
}

void loop() {
 // 1. Reading Sensor Data (Non-blocking timer)
static unsigned long lastSensorRead = 0;
 if (millis() - lastSensorRead > 2000) { // DHT11 needs ~2 seconds between reads
float h = dht.readHumidity();
float t = dht.readTemperature();

if (isnan(h) || isnan(t)) {
SerialBT.println("Failed to read from DHT sensor!");
} else {
SerialBT.print("Temp: ");
```

```
SerialBT.print(t);
 SerialBT.print("C | Humidity: ");
SerialBT.print(h);
SerialBT.println("%");
  }
lastSensorRead = millis();
 }

// 2. Receiving Commands
if (SerialBT.available()) {
command = SerialBT.read();

switch (command) {
 case '0':
myServo.write(0);
SerialBT.println("Servo: 0 degrees");
break;
 case '1':
 myServo.write(90);
SerialBT.println("Servo: 90 degrees");
  break;
case '2':
myServo.write(180);
SerialBT.println("Servo: 180 degrees");
break;
case 'H':
    digitalWrite(ledPin, HIGH);
    SerialBT.println("LED: ON");
  break;
  case 'L':
   digitalWrite(ledPin, LOW);
```

```
  SerialBT.println("LED: OFF");
    break;
   }
  }
 }
```

1. The Arduino IDE was used to write and upload the program to the ESP32.
2. The DHT library was imported to enable temperature and humidity reading from the DHT11 sensor.
3. A Serial (ESP32) interface was configured to handle Bluetooth communication at 115200 baud.
4. The program structure included the following key processes:

Sensor Reading Algorithm:

- Read temperature T and humidity H using:

  T = dht.readTemperature()

  H = dht.readHumidity()

- Validate the data by checking for NaN (invalid readings).

Bluetooth Transmission Algorithm:

- If data is valid, transmit temperature values over Bluetooth using bluetooth.printIn(T).

Command Handling Algorithm:

- The ESP32 continuously checked for incoming Bluetooth commands from the connected smartphone. When a command was received, the microcontroller processed the input and performed the corresponding operation immediately.

  H → Turn LED ON

  L → Turn LED OFF

  0 → Rotate servo motor to 0°

  1 → Rotate servo motor to 90°

  2 → Rotate servo motor to 180°

- After executing each command, the ESP32 transmitted a confirmation message back to the Bluetooth terminal to indicate successful execution of the requested action.

4.3 Bluetooth Pairing and Monitoring

- A smartphone or laptop with Bluetooth capability was used to pair with the ESP32 module.
- A Bluetooth Serial Terminal application was opened to receive data and send commands.
- The device name Bluetooth ID (mobile) was selected to establish the connection.
- Real-time temperature readings were observed as they were transmitted at two-second intervals.

4.4 Sytem Testing Procedure

The system was powered ON, and the serial monitor was observed to verify successful initialization of the ESP32 board, DHT11 sensor, Bluetooth communication, LED, and servo motor.

The surrounding environmental temperature and humidity were monitored continuously through the Bluetooth terminal application. Sensor readings were recorded over a period of time to evaluate the stability and consistency of the collected data.

Several Bluetooth commands were then sent from the smartphone to test the operation of the connected output devices. The LED and servo motor responses were observed carefully to confirm correct functionality.

The following tests were conducted:

- Monitoring real-time temperature and humidity readings
- Turning the LED ON and OFF using Bluetooth commands
- Rotating the servo motor to different angles (0°, 90°, and 180°)
- Verifying two-way Bluetooth communication between the smartphone and ESP32

The experiment was repeated several times to ensure reliable communication, stable sensor readings, and accurate response of the controlled devices.

5.0 DATA COLLECTION

During the experiment, the ESP32 continuously collected temperature and humidity data from the DHT11 sensor and transmitted the readings through Bluetooth communication every 2 seconds. A Bluetooth Serial Terminal application installed on a smartphone was used to monitor the incoming data in real time.

In addition to sensor monitoring, Bluetooth commands were sent from the smartphone to test the operation of the LED and servo motor. The ESP32 successfully received and executed each command, confirming stable two-way Bluetooth communication between the microcontroller and the smartphone.

The following sample data was collected during the experiment under normal room conditions :

Time(s)	Temperature (°C)	Humidity (%)	Command Sent	System Response
0	28.4	71	-	Data received normally
2	28.5	71	-	Data received normally
4	28.6	70	-	Data received normally

6	28.7	70	H	LED turned ON
8	28.7	69	1	Servo rotated to 90°
10	28.8	69	-	Data received normally
12	28.8	68	2	Servo rotated to 180°
14	28.9	68	L	LED turned OFF
16	28.9	68	0	Servo rotated to 0°
18	29.0	67	-	Data steady
20	29.0	67	-	End of sampling

The collected data showed stable and gradual changes in temperature and humidity readings throughout the experiment. The LED and servo motor also responded correctly to all Bluetooth commands sent from the smartphone. These observations confirmed that the ESP32-based wireless monitoring and control system operated reliably and successfully throughout the testing process.

6.0 DATA ANALYSIS

The collected data showed stable and gradual changes in both temperature and humidity readings throughout the experiment. The temperature increased slowly from 28.4°C to 29.0°C over the 20-second sampling period, while the humidity values showed slight decreases over time. These

small variations are normal under indoor environmental conditions and indicate that the DHT11 sensor was functioning properly and providing consistent measurements.

The continuous transmission of sensor readings every 2 seconds demonstrated stable Bluetooth communication between the ESP32 and the connected smartphone. No major interruptions or transmission errors were observed during the monitoring process.

The system also responded successfully to all Bluetooth control commands sent from the smartphone:

- At 6 seconds, the command “H” was transmitted. The ESP32 immediately activated the LED, confirming successful command reception and execution.
- At 8 seconds, the command “1” was sent, and the servo motor rotated to 90° correctly.
- At 12 seconds, the command “2” was transmitted, causing the servo motor to rotate to 180° successfully.
- At 14 seconds, the command “L” was sent, and the LED turned OFF immediately.
- At 16 seconds, the command “0” was transmitted, and the servo motor returned to the 0° position.

The immediate responses of both the LED and servo motor confirmed that the Bluetooth-based control system operated efficiently with minimal delay. Overall, the analysis shows that the ESP32 wireless monitoring system achieved reliable sensor data transmission and accurate remote device control throughout the experiment.

These responses show successful **bidirectional Bluetooth communication**:

1. **Microcontroller → Smartphone**

    - Temperature data arrives every 2 seconds without delay.
2. **Smartphone → Microcontroller**
    - Control commands are received and acted upon in real time.

From the data trend, there was no sudden spike or drop, meaning:

- The Bluetooth transmission was stable.
- The DHT11 sensor provided reliable measurements.
- There was no noticeable noise or misread values.

Overall, the data analysis confirms that the experiment achieved its purpose—accurate temperature monitoring and reliable remote control via Bluetooth.

7.0 RESULTS

The experiment successfully demonstrated wireless temperature monitoring and remote control using Bluetooth communication. The DHT11 sensor provided consistent temperature readings, which were transmitted to the paired smartphone/PC every 1 seconds. The Bluetooth connection remained stable throughout the test, and no transmission errors were observed.

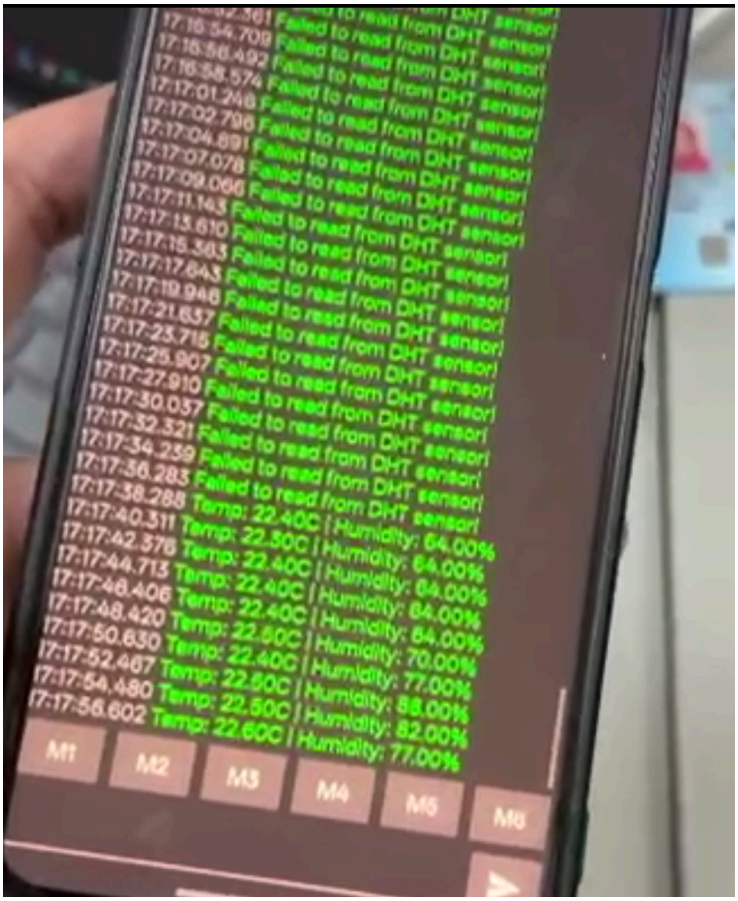
The recorded temperature values ranged from **22.4°C to 29.0°C**, showing a gradual and realistic change in room temperature. This confirms that the sensor readings were stable and accurate.

Additionally, the system successfully responded to user commands sent through the Bluetooth terminal:

- When **"H"** was sent, the microcontroller activated the LED.
- When **"L"** was sent, the output was turned off immediately.
- When **"0"** was sent, the servo moved to 0 degree.
- When **"1"** was sent, the servo moved to 90 degree.
- When **"2"** was sent, the servo moved to 180 degree.

This indicates that **two-way Bluetooth communication** between the microcontroller and the smartphone was functioning correctly.

Overall, the results confirm that the wireless monitoring system worked as intended, providing accurate temperature readings and reliable remote command execution.



7.1 Bluetooth Serial Terminal

8.0 DISCUSSION

8.1 Interpretation of Results

The experiment confirmed that the ESP32-based Bluetooth monitoring and control system operated successfully as intended. The DHT11 sensor was able to measure ambient temperature and humidity accurately, while the ESP32 continuously transmitted the collected readings wirelessly to the connected smartphone through Bluetooth communication at two-second intervals.

The recorded data showed stable and gradual environmental changes, indicating that the sensor readings were consistent and reliable throughout the experiment. In addition, the system responded correctly to all Bluetooth commands sent from the smartphone. Commands used to control the LED and servo motor were executed immediately, confirming successful bidirectional communication between the ESP32 and the mobile device. The experiment therefore validated the effectiveness of the system for wireless monitoring and remote control applications.

8.2 Implications of the Results

The successful operation of the system demonstrates the practicality of using the ESP32 microcontroller for simple Internet of Things (IoT) applications involving wireless monitoring and automation. The experiment showed that low-cost components such as the DHT11 sensor, Bluetooth communication, LED indicators, and servo motors can be integrated effectively to create a functional embedded system.

The ability to monitor environmental conditions remotely and control devices wirelessly suggests potential applications in smart homes, environmental monitoring systems, security systems, and basic industrial automation. The servo motor control feature further expands the project's capability for applications involving automated movement or positioning systems. Overall, the experiment highlights the flexibility and usefulness of ESP32-based wireless control systems in modern embedded technology.

8.3 Discrepancies Between Expected and Observed Outcomes

Although the experiment was generally successful, several minor discrepancies were observed during system operation. Occasionally, the DHT11 sensor produced unstable or delayed readings due to its relatively slow sampling rate and sensitivity to environmental noise. Small fluctuations in humidity values were also observed during data collection.

Minor delays in Bluetooth communication occasionally occurred during reconnection attempts or when the smartphone was positioned farther from the ESP32 module. In some cases, the servo motor movement was slightly inconsistent when insufficient power was supplied directly from the microcontroller. However, these issues did not significantly affect the overall functionality of the system.

Despite these minor discrepancies, the system continued to operate reliably and achieved the main objectives of the experiment successfully.

8.4 Sources of Error and Limitations

Several sources of error and limitations were identified during the experiment. The DHT11 sensor has limited accuracy and a relatively slow response time compared to more advanced

temperature and humidity sensors. As a result, rapid environmental changes may not be detected immediately.

Bluetooth communication is also limited to short-range operation and may experience interference from nearby wireless devices, walls, or obstacles. This can occasionally cause connection instability or communication delays.

Another limitation involves the power requirements of the servo motor. When powered directly from the ESP32, voltage fluctuations may affect motor stability and system performance. An external power supply would improve reliability during continuous operation.

In addition, the system lacked advanced security and error-handling mechanisms. Commands were transmitted as plain text through Bluetooth communication without encryption or authentication, making the system unsuitable for high-security applications.

9.0 CONCLUSIONS

The experiment successfully demonstrated wireless temperature monitoring and remote control through Bluetooth communication using the ESP32 microcontroller. Temperature readings obtained from the DHT11 sensor were accurately displayed on the paired smartphone, confirming that the data transmission process functioned effectively. In addition, the system responded correctly to user commands such as “H”, “L”, “0”, “1”, and “2”. The LEDs were switched ON and OFF accordingly, while the servo motor successfully rotated to 0°, 90°, and 180°, demonstrating proper remote actuation control.

The results verified the initial objective of the experiment, where Bluetooth communication provided a stable and reliable connection between the smartphone and the ESP32. The microcontroller was able to process both outgoing sensor readings and incoming control commands efficiently without transmission errors. Overall, the experiment highlights the effectiveness of Bluetooth technology for small-scale Internet of Things (IoT) applications and demonstrates how wireless communication can improve automation, monitoring, and remote control systems.

10.0 RECOMMENDATIONS

There are several improvements can be made to enhance the wireless temperature monitoring experiment in the future. Based on the experimental setup, the system currently uses an ESP32 microcontroller, a DHT11 temperature and humidity sensor, Bluetooth communication, and a breadboard-based circuit arrangement. Although the experiment successfully demonstrated stable Bluetooth communication and accurate temperature monitoring, the hardware can still be upgraded to improve performance and reliability. For instance, the DHT11 sensor may be replaced with more accurate sensors such as the DHT22 or BME280, which also provide better humidity measurement capabilities. In addition, instead of only controlling LEDs and servo motors, a real fan together with a motor driver such as the L298N could be integrated to create a more realistic temperature control application.

The software and communication system can also be improved by developing a custom mobile application using MIT App Inventor or Flutter. The application could display real-time temperature readings in graphical form, provide dedicated FAN ON/OFF buttons, and store

sensor data automatically for monitoring purposes. Since the experiment showed successful two-way Bluetooth communication where commands such as “H”, “L”, “0”, “1”, and “2” controlled LEDs and servo motor positions, future improvements may include adding acknowledgment messages such as “Command received” and implementing error handling for invalid inputs to increase system robustness.

Furthermore, the results showed that the recorded temperature values ranged from 22.4°C to 29.0°C with stable transmission every one second. To improve data analysis and presentation, Python or MATLAB can be used to generate live temperature graphs and save the collected data in CSV format for future reference and analysis. Additional smart control features can also be implemented, such as automatic fan activation based on temperature thresholds, buzzer alerts for high temperatures, OLED displays for direct monitoring, and menu navigation systems. These enhancements would make the wireless monitoring system more interactive, reliable, and suitable for advanced real-world applications.

11.0 REFERENCES

Arduino Bluetooth Basic Tutorial. (2018). Arduino Project Hub.

https://projecthub.arduino.cc/NeilChaudhary/arduino-bluetooth-basic-tutorial-9cff12

Basic Bluetooth communication with Arduino & HC-05. (2021). Arduino Project Hub.

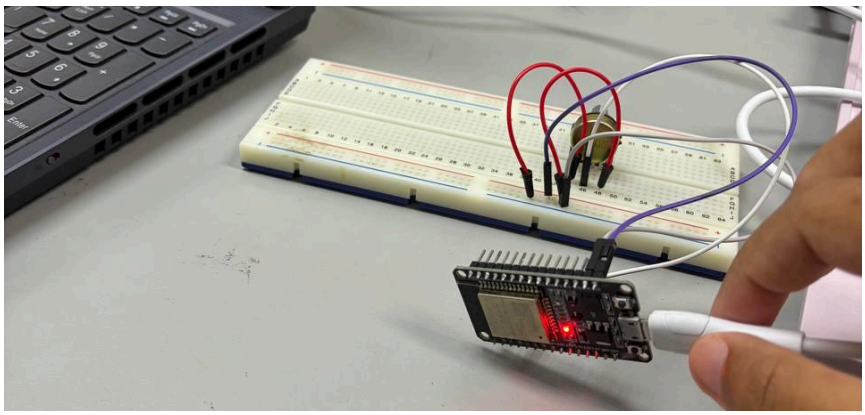
https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-05-3a431c

Dejan. (2019, February 8). *Arduino and HC-05 Bluetooth Module Tutorial - HowToMechatronics*. HowToMechatronics.

https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/

APPENDICES

Appendix A (Hardware Test Photo)



Picture 12.0

ACKNOWLEDGEMENTS

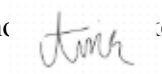
The group would like to express our deepest appreciation to our laboratory instructor, Dr. -Ing. Wahju Sediono for the continuous guidance, encouragement, and clear explanations provided throughout the course of this experiment. His expertise and detailed feedback greatly enhanced our understanding of digital logic systems and circuit interfacing, especially in applying theoretical knowledge to practical implementation. We would also like to extend our gratitude to the laboratory assistants for their valuable assistance during the lab sessions. Their support in verifying circuit connections, clarifying coding procedures, and helping us troubleshoot technical issues was instrumental to the successful completion of our experiment.

In addition, we would like to thank our classmates and also group members, Anis Aqilah Binti Abdullah, Amir Aiman bin Ariff Affendi and Azril Ulwan Bin Mohammad Imri for their cooperation and willingness to share insights, suggestions, and resources during the experiment. Collaborative discussions and teamwork played a crucial role in identifying and resolving problems efficiently. Lastly, we appreciate the opportunity provided by the Department of Mechatronics Engineering to conduct this experiment, which allowed us to strengthen our practical skills in Arduino programming, electronic interfacing, and logical circuit design. The

collective guidance and support from all parties involved contributed significantly to the success and learning outcome of this project.

STUDENTS' DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgment, and that the original work contained herein has n  en or done by unspecified sources or persons. We hereby certify that this report **has not been done by only one individual**, and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall b  ted for **marking**, and this **final printed report** has been **verified by us**.

Signature:  Name: Ami Bin Ariff Affendi Matric Number: 2315925 Contributions: Introduction, Materials and Equipment, Experimental setup, Results	Read [/] Understand [/] Agree [/]

Signature: Name: Anis Aqilah Binti Abdullah Matric Number: 2329678 Contributions: Abstract, Conclusion, Recommendations, References, Appendices, Acknowledgements	Read [/] Understand [/] Agree [/]
Signature: Name: Azril Ulwan Bin Mohammad Imri Matric Number: 2314617 Contributions: Methodology, Data Collection, Data Analysis, Discussions	Read [/] Understand [/] Agree [/]